EVENTUS

Group 4: Damien, Denzel, Joash, Jian Xin, Andrea, Gui Yun

SCHEMA DESIGN

users

_id (ObjectId)	name (String)	email (String)	password (String)	phone (Number)	about (String)

events

_id (ObjectId)	Title (String)	Venue (String)	Description (String)	Category (String)	startDate (Date)	endDate (Date)	maxParticipants (Number)	owner (ref: "User")

favourites

_id (ObjectId)	user (ref: “User”)	event (ref: “Event”)	notes (String)	reminder (Boolean)	tag (String, enum: ["", "social", "educational", "professional"]

rsvps

_id (ObjectId)	user (ref: “User”)	event (ref: “Event”)	status (String, enum: [“confirmed”, “waitlist”])	notes (String)	timestamps (createdAt, updatedAt)

rsvpactivities

rsvp (ref: “RSVP”)	event (ref: “Event”)	user (ref: “User”)	action (String, enum: [cancelled,waitlist]	timestamps (createdAt, updatedAt)

todos

user (ref: “User”)	title (String)	priority (String, enum: ["low", "medium", "high"]	deadline (Date)	status (String, enum: ["pending", "completed"])

attendances

user (ref: “User”)	event (ref: “Event”)	checkedInAt (Date)	qrCodeValue (String)	ownerNotes (String)

FEATURE 1: SECURITY (AUTH)

Overview

Implements user registration, login, logout, password hashing, session management, route protection, and full user profile management

Main Views

- Main Views
- Home Page
- Event Details

CRUD Operations

Create

POST /auth/signup

Create new User with bycrypt-hashed password, 10 salted rounds using Unique CSPRNG salt per user

Read

POST /auth/login

Looks up user by email, verifies password hash

GET /profile

Renders logged-in user's profile

Update

POST /profile/update

Updates name, email, phone, about

POST /profile/password

Verifies old password, saves new password hash

Delete

POST profile/delete

Verifies password, cascade-deletes account and all associated data

FEATURE 1: SECURITY (AUTH)

Functions

Session cookies

- httpOnly: true - prevents XSS attacks
- saveUninitialized: false - no session for unauthenticated users
- maxAge: 24h - automatic expiry

Middlewares

Called when necessary in routes to enforce access control

- requireAuth: must be logged in

else redircts to /auth/login

- requireEventOwner: must own resource
- requireHasEvent: must hace created events

Account deletion

Cascade hard-deletes own upcoming/ongoing events + child records

Anonymize past owned events

- RSVPs and check-ins preserved on attendee's side, but organiser changed to "Deleted User"

Deletes upcoming/ongoing RSVPs

- Frees up confirmed spots, opens up for waitlist

Anonymize past RSVPs

- Preserves record for owner as "Deleted User"

FEATURE 2: EVENTS

Overview

Let users create, browse, and manage events, with owners having full control to edit and delete their own events.

Main Views

- Home Page
- Event Details
- Create event page
- Update event page

CRUD Operations

Create

*POST /events

Creates a new event and saves it with the owner's userid

Read

GET /events & GET /

List all events with optional search, filter, sort

*GET /events/new

Loads event creation form

GET /events/:id

Shows single event info with organiser info

**GET /events/:id/edit

Loads pre-populated event form for edits

Update

**POST /events/:id/update

Re-validates form and saves updated info, auto-promotes waitlisted participants if capacity increased

Delete

**POST /events/:id/delete

Deletes event and cascades to all affected schemas

***uses requireAuth middleware**

****uses requireAuth and requireEventOwner middleware**

Error Handling

Form Validation

- Title required, min 3 char
- Venue required
- Description required, min 10 char
- Category required
- Start Date required, the next day onwards
- End Date required, after Start Date
- Max Participants required, whole number, >= 1

Invalid form re-renders with errors and inputs preserved

Feature Logic

Increasing capacity automatically promotes waitlisted participants

All routes catch errors with req.flash and redirect safely

FEATURE 3: FAVOURITES

CRUD Operations *RequireAuth*

Create

POST /events/:id/favourites

Adds to favourites schema

Read

GET /favourites

Displays all events that was added to favourites

Update

POST /favourites/:id/notes

POST /favourites/:id/reminder

User can add notes for themselves or reminders when the event draws near

Delete

POST /favourites/:id/delete

User can remove event from favourites

Form Validation

User

Required

Add to favourites button will not be shown without logging in

Event

User cannot add their own event to favourites

User cannot add an event that was already favourited

Notes

Optional, defaults to “”

Reminder

Optional, defaults to false

Reminders will show when user logs in

Tag

Optional, enum: social, educational, professional

FEATURE 4: RSVP

Overview

A seamless event registration system featuring real-time capacity tracking and automated waitlist management.

Main Views

- My events (RSVPs)

CRUD Operations

Create

POST /events/:id/rsvp

Creates a new RSVP entry ('confirmed' or 'waitlisted')

Update

POST /events/:id/cancel

Auto promotes earliest waitlisted user to 'confirmed' (FIFO by createdAt) if a 'confirmed' user cancels their RSVP

Read

GET /events/my

Fetches all RSVPs for the logged-in user

Delete

POST /events/:id/cancel

Logs cancellation to RSVPActivity before deleting RSVP entry

***all routes uses the requireAuth middleware**

Error Handling

Form Validation

- Event owners cannot RSVP to their own events
- Prevents duplicate RSVP entries
- Notes cannot exceed 100 characters

Form Feedback

- Uses req.flash to provide instant UI alerts for success, waitlist status, or invalid actions.

Feature Logic

- A 'confirmed' user cancelling their RSVP automatically promotes the next waitlisted attendee.

FEATURE 5: OWNER DASHBOARD

Overview

Central owner workspace for analytics, event management, attendance, and personal tasks

Main Views

- Dashboard analytics
- My Events
- Participants list/ waitlist (RSVP List)
- Attendance + CSV export
- To-Do list

Error Handling

Todo Validation

- Title required
- Priority must be low, medium, or high
- Deadline cannot be before today

Form Feedback

Invalid todo forms re-render with errors and preserved input values

Missing Data

- Missing event or todo shows flash error
- User is redirected to a safe dashboard page

Server Errors

- req.flash("error", ...) for user feedback
- Catch blocks log errors and redirect safely

Feature Logic

Deleting a confirmed RSVP auto-promotes the next waitlisted attendee, reducing manual admin work

FEATURE 5: OWNER DASHBOARD

CRUD Operations

Create

POST /todos/create
Creates a new personal dashboard task

Read

*GET /dashboard
Stats + top upcoming + merged activity feed

*GET /dashboard/myevents
Ongoing, upcoming, past event sections

*GET /dashboard/todo
Todo list sorted by deadline or priority

*GET /dashboard/events/:id/participants
Reads confirmed + waitlisted RSVPs

*GET /dashboard/events/:id/attendance
Displays check-in status and owner notes

*GET /dashboard/events/:id/attendance/export
Exports attendance as CSV

GET /todos/:todoId/edit
Loads edit form for an existing todo

Update

**POST /dashboard/events/:eventId/participants/:rsvpId/promote
Promotes waitlist to confirmed

**POST /dashboard/events/:eventId/attendance/:userId/notes
Updates private owner notes

POST /todos/:todoId/update
Updates title, priority, or deadline

POST /todos/:todoId/toggle
Toggles pending/completed status

Delete

POST /dashboard/events/:eventId/participants/:rsvpId/delete
Deletes RSVP and can auto-promote next waitlisted

POST /todos/:todoId/delete
Removes a task from the todo list

***all routes use the requireAuth middleware**
***routes labelled with a "*" use an additional requireHasEvents middleware**
***routes labelled with a "**" use an additional requireEventOwner middleware**

FEATURE 6: CHECK IN

Overview

Lets event owners manage real-time check-ins via QR scanning or manual lookup, with controls to view attendance, edit timestamps, update notes, and export data.

CRUD Operations

All routes needs RequireAuth

Create

POST /checkin/:eventId
Manually check in an attendee

POST /checkin/:eventId/qr
Check in via QR code scanner

Read

GET /checkin/:eventId
Load the check-in dashboard

GET /checkin/:eventId/export
Download attendee list as CSV

Update

PATCH /checkin/:eventId/update-time
Edit a check-in timestamp

PATCH /checkin/:eventId/owner-notes
Save private notes for attendee

POST /checkin/:eventId/update-rsvp
Change attendee's RSVP status

Delete

POST /events/:id/delete
Undo a check-in

Form Validation

Identifier
Required if no userId

userID
Valid ObjectID

eventID
Valid ObjectID

qrValue
must be “UserID:eventID”

checkedInAt
Parseable Date String

ownerNotes
Trimmed, Max 500 chars

status
“confirmed” or “waitlist”

THANK YOU